PEMROGRAMAN BERORIENTASI OBJEK

Pengenalan Paradigma, Sintaks Dasar Java, dan Proses Kompilasi

Minggu 1 — Tahun Akademik 2026/2027

```java
public class Mahasiswa {
  String nama;
  int semester;

  void belajar() {
    System.out.println(
      "Siap ngoding!");
  }
}
```

D4 Teknologi Rekayasa Perangkat Lunak
Fakultas Vokasi — Institut Teknologi Del

Cynthia Deborah Nababan, M.Kom.

AGENDA MINGGU 1

Apa yang Akan Kita Pelajari?

1

Mengapa Paradigma Itu Penting

Menelusuri akar pendekatan prosedural menuju berorientasi objek.

2

Konsep Dasar OOP

Objek, kelas, atribut, dan method melalui studi kasus nyata.

3

Mengenal Java

JDK, JRE, JVM, dan mengapa Java masih relevan hingga kini.

4

Sintaks & Kompilasi

Struktur program Java sederhana dan proses javac → bytecode → JVM.

5

Praktik Langsung

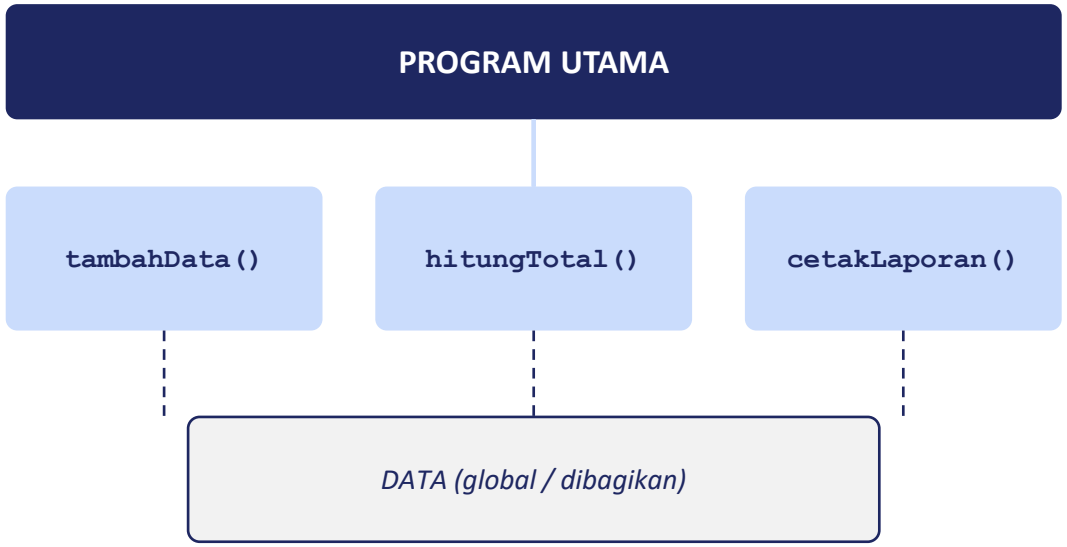
Menulis, mengompilasi, dan menjalankan program Java pertamamu.

LATAR BELAKANG

Sebelum OOP: Pendekatan Prosedural

Program disusun sebagai **serangkaian langkah (fungsi)** yang dieksekusi berurutan — data dan fungsi dikelola secara terpisah.

- Dekomposisi masalah: data + fungsi
- Alur pemikiran top-down
- Cocok untuk program kecil dan linear
- Sulit dipelihara saat sistem membesar — data “bebas” diakses oleh banyak fungsi



Semua fungsi bisa mengakses & mengubah data yang sama.

STUDI KASUS

Sistem Informasi Rumah Sakit — Gaya Prosedural

Fungsi dikelompokkan per modul, namun tetap bekerja di atas data bersama yang mudah tercampur antar-modul.

Modul Pasien	Modul Klinik	Modul Apotek

`tambahPasien()`
`ubahPasien()`
`cariRiwayat()`
	`kelolaTindakan()`
	`buatLaporanTindakan()`
		`kelolaObat()`
		`cariObat()`
		`cetakLaporan()`

Data pasien, obat, dan tindakan tersimpan terpisah — relasi antar-data dikelola manual oleh programmer.

KONTEKS HISTORIS

Lahirnya Pemrograman Berorientasi Objek

Software yang makin kompleks butuh cara baru memodelkan dunia nyata — sebagai kumpulan objek yang saling berinteraksi.

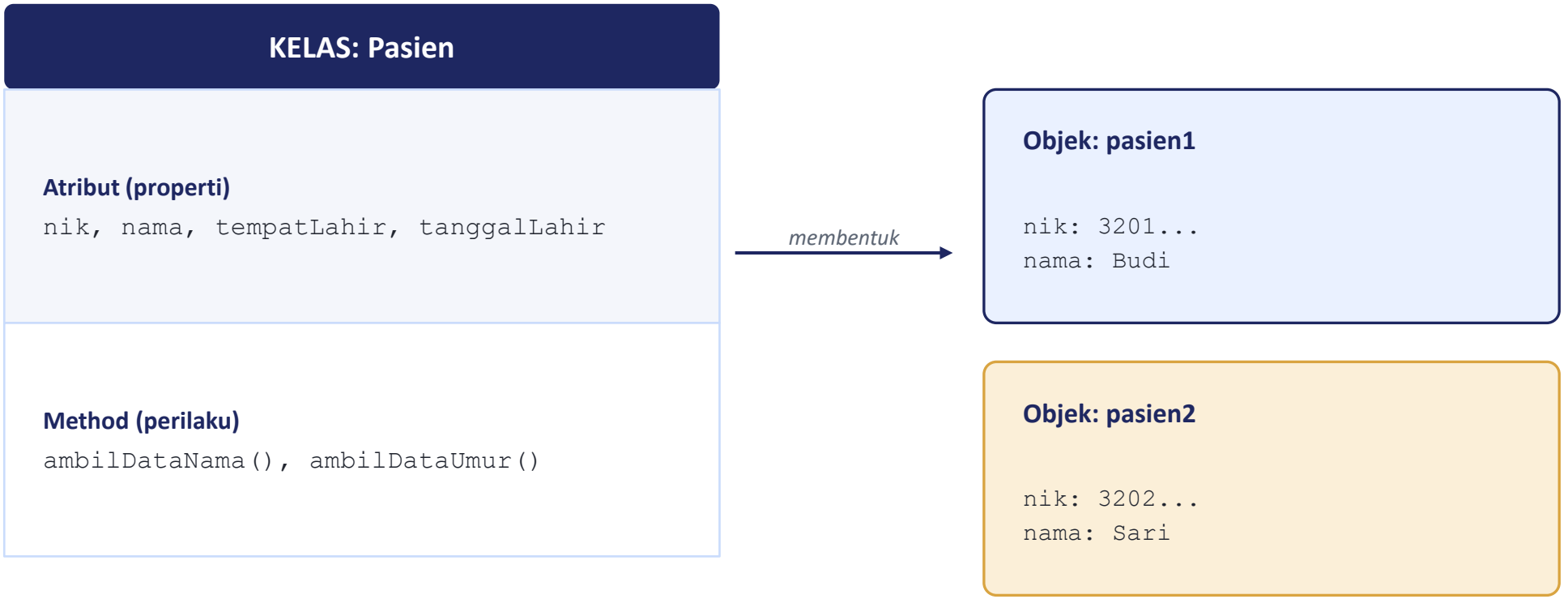


Sumber: Dahl & Nygaard (Turing Award 2001); History of Programming Languages II.

KONSEP INTI

Objek, Kelas, Atribut, dan Method

OOP memandang sistem sebagai kumpulan objek — setiap objek punya identitas, data, dan perilaku.



PERBANDINGAN

Prosedural vs Berorientasi Objek

Studi kasus: Sistem Informasi Perpustakaan

Aspek	Prosedural	Berorientasi Objek
Fokus	Fungsi / prosedur yang dijalankan	Objek yang berinteraksi
Contoh unit	meminjamBuku(), hitungDenda()	Buku, Anggota, Petugas, Laporan
Data & logika	Terpisah, dapat diakses banyak fungsi	Digabung dalam satu objek (encapsulation)
Skalabilitas	Menyulitkan saat sistem membesar	Lebih mudah dipelihara & dikembangkan

RELEVANSI SAAT INI

OOP: Masih Menjadi Fondasi Utama

Hampir seluruh bahasa dan platform besar hari ini tetap dibangun di atas konsep objek dan kelas.

Java

Android, backend enterprise, sistem perbankan

C# / .NET

Aplikasi desktop, game (Unity), layanan Microsoft

Python

Data science & AI — semua tetap berbasis class

Kotlin & Swift

Aplikasi mobile Android & iOS modern

Menguasai konsep objek dan kelas berarti membuka pintu ke hampir semua ekosistem pengembangan software modern, bukan cuma satu bahasa.

JDK, JRE, dan JVM

JDK

Java Development Kit

Paket lengkap untuk developer: compiler (javac), tools, dan JRE di dalamnya.

JRE

Java Runtime Environment

Lingkungan untuk menjalankan program Java: berisi JVM + library standar.

JVM

Java Virtual Machine

Mesin virtual yang mengeksekusi bytecode — kunci “write once, run anywhere”.

Instalasi: unduh JDK (bukan hanya JRE) dari distribusi resmi (Oracle JDK atau OpenJDK / Adoptium) sebelum praktik.

DARI KODE KE EKSEKUSI

Proses Kompilasi Program Java



Mengapa bytecode? File .class tidak bergantung pada sistem operasi — selama ada JVM, program yang sama bisa berjalan di Windows, Linux, atau macOS. Inilah filosofi “Write Once, Run Anywhere”.

```
$ javac HelloWorld.java     // mengompilasi ke HelloWorld.class
$ java HelloWorld            // JVM menjalankan bytecode
Hello, World!
```

Anatomi Program Java Sederhana

```
public class HelloWorld {

  public static void main(
      String[] args) {

    System.out.println(
      "Hello, World!");

  }
}
```

- **`public class HelloWorld`**

  Nama file HARUS sama dengan nama kelas: HelloWorld.java

- **`public static void main`**

  Titik masuk (entry point) — JVM selalu mulai dari sini

- **`String[] args`**

  Menampung argumen dari command line

- **`System.out.println(...)`**

  Mencetak teks ke layar, diakhiri titik koma

Catatan: Java bersifat case-sensitive — huruf besar/kecil dibedakan, termasuk pada nama kelas dan file.

PRAKTIK LANGSUNG · 01

Menulis, Mengompilasi, dan Menjalankan Hello World

1. Buka teks editor / IDE, buat file baru bernama HelloWorld.java
2. Ketik ulang kode HelloWorld dari slide sebelumnya (jangan copy-paste!)
3. Buka terminal, arahkan ke folder file tersebut
4. Jalankan: javac HelloWorld.java → memunculkan HelloWorld.class
5. Jalankan: java HelloWorld → lihat hasilnya di layar

Tantangan kecil: ganti teks “Hello, World!” dengan namamu, lalu kompilasi ulang — tanpa melihat catatan.

PRAKTIK LANGSUNG · 02

Membuat & Menggunakan Objek Pertamamu

```
public class Mahasiswa {
  String nama;
  int semester;


  void perkenalan() {
    System.out.println(
      "Saya " + nama +
      ", semester " + semester);
  }


  public static void main(
      String[] args) {
    Mahasiswa m1 = new Mahasiswa();
    m1.nama = "Rina";
    m1.semester = 1;
    m1.perkenalan();
  }

}
```

Perhatikan:

- class Mahasiswa = cetak biru (kelas)
- new Mahasiswa() = membuat objek baru
- m1.nama, m1.semester = mengisi atribut
- m1.perkenalan() = memanggil method objek

Coba sendiri:

Tambahkan objek m2 dengan data berbeda, lalu panggil perkenalan() untuk m2. Bandingkan hasilnya dengan m1 — satu kelas, banyak objek berbeda.

SEBELUM PERTEMUAN BERIKUTNYA

Tantangan Mandiri: Mengulik Lebih Jauh

1 Eksplorasi

Ubah atribut & tambah method baru pada kelas Mahasiswa. Misalnya: hitungSKS() atau ubahSemester().

2 Rasa Ingin Tahu

Cari tahu: apa yang terjadi jika nama file berbeda dengan nama kelas publik? Coba dan amati error-nya.

3 Koneksi Konsep

Bandingkan versi prosedural dan OOP dari kasus “Menambah Data Pasien”, apa perbedaan nyata di kode program?

Semangat mengulik kode adalah modal utama menguasai OOP — error bukan kegagalan, tapi petunjuk untuk belajar lebih dalam.

REFERENSI

Sumber Bacaan & Rujukan

- **Materi dasar (diadaptasi)**

  Materi kuliah OOSD, IT Del — cis.del.ac.id

- **Oracle — The Java Tutorials: Getting Started**

  docs.oracle.com/javase/tutorial/getStarted/cupojava

- **A Brief History of Object-Oriented Programming**

  web.eecs.utk.edu/~bvanderz/cs302/notes/oo-intro.html

- **The Origins of Object Orientation**

  ootips.org/history.html

Terima Kasih

Ada pertanyaan tentang paradigma, sintaks, atau kompilasi Java?

D4 Teknologi Rekayasa Perangkat Lunak 2026/2027 — Fakultas Vokasi, Institut Teknologi Del